

UNIVERSITÀ DELLA CALABRIA

Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e  
Sistemistica (DIMES)

Corso di Laurea Magistrale in Ingegneria Elettronica

**Corso di Progettazione di Sistemi Digitali**

# **Progettazione e Implementazione Hardware di un Filtro Laplaciano 3x3 con Aritmetica Ottimizzata su FPGA**

*Candidati:*

Francesco Di Bruno (Matr. 281109)

Mariafrancesca Lamerata (Matr. 283579)

Rosa Marino (Matr. 281382)

Luca Gaetano Sansalone (Matr. 281369)

*Docenti:*

Prof.ssa Stefania Perri

Prof.ssa Fanny Spagnolo

Anno Accademico 2025/2026

# Indice

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                           | <b>1</b> |
| 1.1      | Contesto e Obiettivi . . . . .                | 1        |
| 1.2      | Specifiche di Progetto . . . . .              | 1        |
| <b>2</b> | <b>Cenni Teorici</b>                          | <b>2</b> |
| 2.1      | La Convoluzione Spaziale . . . . .            | 2        |
| 2.2      | Il Filtro Laplaciano . . . . .                | 2        |
| 2.3      | Moltiplicazione: Algoritmo di Booth . . . . . | 3        |
| 2.4      | Adder Tree e Pipeline . . . . .               | 3        |
| <b>3</b> | <b>Architettura del Sistema</b>               | <b>4</b> |
| 3.1      | Schema a Blocchi Top-Level . . . . .          | 4        |
| 3.2      | Gestione della Finestra Mobile . . . . .      | 5        |
| <b>4</b> | <b>Implementazione VHDL - Versione 1</b>      | <b>6</b> |
| 4.1      | RCA e Full Adder . . . . .                    | 6        |
| 4.2      | BoothMultiplier . . . . .                     | 6        |
| 4.3      | AdderTree . . . . .                           | 7        |
| 4.4      | BufferLine (Top Level e FSM) . . . . .        | 7        |
| <b>5</b> | <b>Verifica e Simulazione</b>                 | <b>8</b> |
| 5.1      | Strategia di Test (Testbench) . . . . .       | 8        |
| 5.1.1    | Generazione degli Stimoli . . . . .           | 8        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 5.1.2    | Analisi Teorica dei Risultati Attesi . . . . .                      | 9         |
| 5.1.3    | Tabella Riepilogativa dei Casi di Test . . . . .                    | 10        |
| 5.2      | Risultati della Simulazione . . . . .                               | 11        |
| 5.2.1    | Fase di Reset e Inizializzazione . . . . .                          | 11        |
| 5.2.2    | Elaborazione delle Aree Uniformi . . . . .                          | 11        |
| 5.2.3    | Rilevamento dei Bordi . . . . .                                     | 12        |
| <b>6</b> | <b>Implementazione VHDL - Versione 2</b>                            | <b>13</b> |
| 6.1      | Motivazione e Strategia di Ottimizzazione . . . . .                 | 13        |
| 6.2      | LaplacianFilter_v2: Logica Hardcoded e CLA . . . . .                | 13        |
| 6.3      | AdderTree_v2: Albero CLA . . . . .                                  | 14        |
| 6.4      | Vincoli XDC e Pblock . . . . .                                      | 14        |
| 6.5      | Modifica del Bus di Uscita . . . . .                                | 16        |
| <b>7</b> | <b>Implementazione VHDL - Versione 3</b>                            | <b>17</b> |
| 7.1      | Motivazione: Il Carry Save Adder per Somme Multi-operando . . . . . | 17        |
| 7.2      | LaplacianFilter_v3: RCA per la Negazione . . . . .                  | 17        |
| 7.3      | AdderTree_v3: Albero CSA . . . . .                                  | 18        |
| <b>8</b> | <b>Risultati post-implementazione e Confronto tra le Versioni</b>   | <b>19</b> |
| 8.1      | Timing . . . . .                                                    | 19        |
| 8.1.1    | Sintesi dei Risultati Globali . . . . .                             | 19        |
| 8.1.2    | Analisi del Critical Path . . . . .                                 | 20        |
| 8.1.3    | Reset e Distribuzione del Clock . . . . .                           | 20        |
| 8.1.4    | Nota sul Worst Hold Slack . . . . .                                 | 20        |
| 8.2      | Utilizzo delle Risorse . . . . .                                    | 21        |
| 8.2.1    | Riduzione delle risorse dalla v1 alla v2 . . . . .                  | 21        |
| 8.2.2    | Variazioni nella v3 rispetto alla v2 . . . . .                      | 21        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 8.3      | Potenza . . . . .                                    | 22        |
| 8.4      | Discussione Critica dei Risultati . . . . .          | 22        |
| 8.4.1    | Trade-off Timing vs. Area nella v3 . . . . .         | 22        |
| 8.4.2    | Convergenza dell'Ottimizzazione in v1 e v2 . . . . . | 22        |
| 8.4.3    | Ruolo del Pblock . . . . .                           | 23        |
| <b>9</b> | <b>Conclusioni</b>                                   | <b>24</b> |

# Capitolo 1

## Introduzione

### 1.1 Contesto e Obiettivi

Nel campo dell'elaborazione digitale dei segnali e delle immagini, la convoluzione bidimensionale rappresenta una delle operazioni più onerose dal punto di vista computazionale. L'implementazione software su CPU general-purpose soffre spesso di colli di bottiglia dovuti all'accesso sequenziale alla memoria. Le architetture FPGA (Field Programmable Gate Array), grazie alla loro natura parallela, permettono di accelerare drasticamente tali operazioni.

L'obiettivo di questo progetto è progettare un componente hardware in grado di ricevere un flusso video seriale (pixel by pixel), applicare un filtro Laplaciano per il rilevamento dei contorni e fornire in uscita il pixel elaborato in tempo reale.

### 1.2 Specifiche di Progetto

Il sistema deve rispettare i seguenti requisiti:

- **Input:** Immagine 32x32 pixel, 8 bit per pixel (Grayscale).
- **Kernel:** Finestra 3x3 con coefficienti fissi (Laplaciano).
- **Aritmetica:** Implementazione custom (non comportamentale standard).
  - Moltiplicatori basati su codifica di Booth.
  - Somma tramite Ripple Carry Adders (RCA) organizzati ad albero.
- **Interfaccia:** Protocollo di handshake AXI-Stream (`tvalid`, `tready`, `tlast`).
- **Output:** Dato espanso a 24 bit per gestire il range dinamico e il segno.

# Capitolo 2

## Cenni Teorici

### 2.1 La Convoluzione Spaziale

Il filtraggio spaziale è una tecnica fondamentale nell'Image Processing. Data un'immagine  $I(x, y)$  e una maschera (o kernel)  $K$  di dimensioni  $M \times N$ , la convoluzione discreta produce un'immagine di uscita  $O(x, y)$  secondo la formula:

$$O(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b K(i, j) \cdot I(x - i, y - j) \quad (2.1)$$

Nel nostro caso, con una finestra  $3 \times 3$ , ogni pixel di uscita è il risultato della somma pesata dei 9 pixel circostanti. Questo richiede che l'hardware abbia accesso simultaneo a 3 righe dell'immagine.

### 2.2 Il Filtro Laplaciano

Il Laplaciano è un operatore di derivata seconda, isotropo (invariante alla rotazione), utilizzato per evidenziare le transizioni rapide di intensità (bordi). La versione discreta utilizzata in questo progetto impiega la seguente maschera  $3 \times 3$ :

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (2.2)$$

La somma dei coefficienti è zero ( $\sum K = 0$ ). Ciò implica che in aree uniformi dell'immagine (dove  $I(x, y)$  è costante), l'uscita del filtro sarà nulla. In corrispondenza di un gradino (bordo), l'uscita presenterà un passaggio per lo zero con picchi positivi e negativi.

## 2.3 Moltiplicazione: Algoritmo di Booth

Per implementare i 9 moltiplicatori necessari, si è scelto l'algoritmo di Booth (Radix-4). Questo metodo riduce il numero di prodotti parziali necessari dimezzandoli rispetto a una moltiplicazione standard "shift-and-add". L'algoritmo esamina terne di bit del moltiplicatore e determina l'operazione da compiere sul moltiplicando ( $A$ ):

| $b_{i+1}$ | $b_i$ | $b_{i-1}$ | Operazione |
|-----------|-------|-----------|------------|
| 0         | 0     | 0         | +0         |
| 0         | 0     | 1         | + $A$      |
| 0         | 1     | 0         | + $A$      |
| 0         | 1     | 1         | + $2A$     |
| 1         | 0     | 0         | - $2A$     |
| 1         | 0     | 1         | - $A$      |
| 1         | 1     | 0         | - $A$      |
| 1         | 1     | 1         | -0         |

Tabella 2.1: Tabella di codifica Booth Radix-4

Questa tecnica è particolarmente efficiente in hardware poiché le operazioni di shift ( $2A$ ) e complemento ( $-A$ ) sono computazionalmente economiche.

## 2.4 Adder Tree e Pipeline

Sommare sequenzialmente i 9 risultati dei moltiplicatori creerebbe un percorso critico molto lungo, limitando la frequenza di clock massima. Si è optato per un **Adder Tree** (Albero di Somma). L'albero riduce la profondità logica a  $\lceil \log_2 9 \rceil = 4$  stadi. Inserendo registri di pipeline tra ogni stadio di somma, è possibile ottenere un throughput di 1 pixel per ciclo di clock, pagando un prezzo in termini di latenza iniziale (riempimento della pipeline).

# Capitolo 3

## Architettura del Sistema

### 3.1 Schema a Blocchi Top-Level

L'architettura del sistema **BufferLine** è progettata per elaborare i dati in streaming. Lo schema a blocchi principale comprende:

1. **Line Buffers:** Due registri a scorrimento (o RAM) di lunghezza pari alla larghezza dell'immagine ( $N = 32$ ) meno la dimensione del kernel. Servono a ritardare i dati per allineare verticalmente i pixel.
2. **Window Generator:** Registri che espongono simultaneamente la finestra  $3 \times 3$  ( $D_{00} \dots D_{22}$ ).
3. **Filter Unit:** Blocco contenente i 9 moltiplicatori di Booth in parallelo.
4. **Adder Tree:** Rete di sommatore pipelined che calcola il risultato finale.
5. **Control FSM:** Macchina a stati per gestire i segnali di handshake AXI.

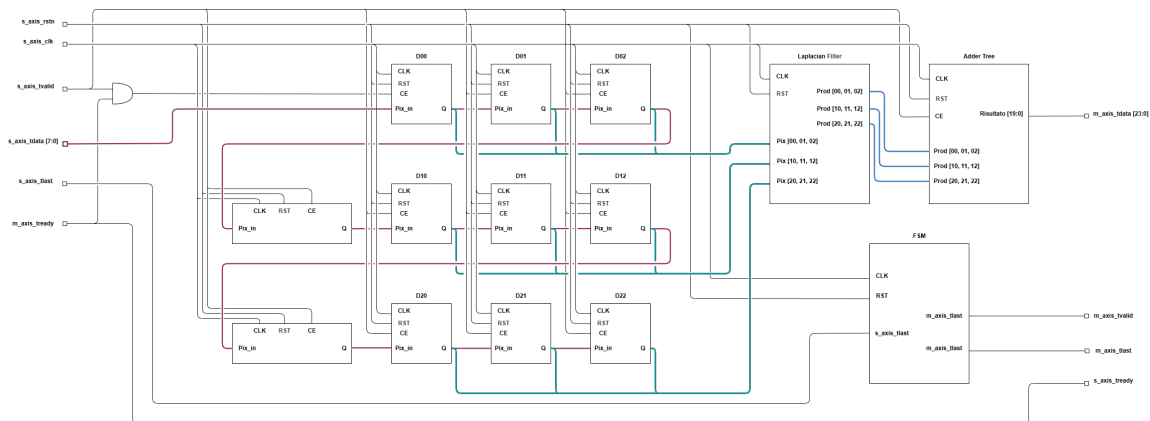


Figura 3.1: Schema a blocchi dell'architettura hardware progettata.



## 3.2 Gestione della Finestra Mobile

L'immagine arriva pixel per pixel. Per calcolare la convoluzione al pixel  $(x, y)$ , sono necessari i pixel delle righe  $y - 1$ ,  $y$  e  $y + 1$ . I Line Buffer immagazzinano le righe precedenti. Quando il dato  $D_{00}$  entra nel sistema, il dato  $D_{10}$  esce dal primo buffer e il dato  $D_{20}$  esce dal secondo buffer, garantendo l'allineamento colonna.

# Capitolo 4

## Implementazione VHDL - Versione 1

In questo capitolo vengono descritti i dettagli implementativi dei componenti principali. Il codice completo è riportato in Appendice.

### 4.1 RCA e Full Adder

Alla base dell'aritmetica vi è il **FullAdder**, componente combinatorio che calcola la somma di 3 bit. Il componente **RCA** (Ripple Carry Adder) istanzia una catena parametrica di Full Adder. Sebbene più lento di un Carry Look-ahead Adder, l'RCA offre un'implementazione semplice. È stato implementato con generics per adattarsi alle diverse larghezze di bit richieste nell'albero di somma (da 16 a 20 bit).

### 4.2 BoothMultiplier

Questa entità realizza la logica di moltiplicazione.

- **Input:** Pixel (8 bit unsigned), Coefficiente (4 bit signed).
- **Generazione parziali:** Vengono calcolati internamente i segnali **A\_pos**, **A\_neg** (tramite complemento a 2), **A2\_pos** (shift sx di 1) e **A2\_neg**.
- **Selezione:** Due processi **CASE** implementano la tabella di Booth per generare due prodotti parziali (**PP0** e **PP1**).
- **Somma:** **PP1** viene shiftato di 2 posizioni a sinistra (peso  $2^2$ ) e sommato a **PP0**.

## 4.3 AdderTree

L'albero di somma gestisce la crescita dei bit (Bit Growth) per evitare overflow senza troncamento.

- **Livello 1:** Somma coppie di prodotti (16 bit  $\rightarrow$  17 bit).
- **Livello 2:** Somma risultati precedenti (17 bit  $\rightarrow$  18 bit).
- **Livello 3 e 4:** Somme finali fino a 20 bit.

**Nota sulla sincronizzazione:** Poiché l'albero è binario, il nono input ( $P8$ ) rimane "orfano" nei primi stadi. È stato inserito un registro a scorrimento ( $r\_p8\_v1 \dots v3$ ) per ritardare  $P8$  e allinearlo temporalmente all'ultimo stadio di somma.

## 4.4 BufferLine (Top Level e FSM)

La Control Unit è una FSM a 4 stati:

1. **S0 (Reset):** Inizializzazione registri.
2. **S1 (Pre-filling):** Accetta dati in ingresso ma non valida l'uscita finché i buffer di linea non sono pieni.
3. **S2 (Processing):** Elaborazione a regime.  $m\_axis\_tvalid$  è alto.
4. **S3 (Flushing):** Dopo aver ricevuto l'ultimo pixel ( $tlast$ ), continua a processare per svuotare la pipeline aritmetica.

# Capitolo 5

## Verifica e Simulazione

### 5.1 Strategia di Test (Testbench)

La verifica funzionale del sistema è stata condotta mediante un Testbench che genera un'immagine sintetica di dimensioni  $32 \times 32$  pixel. L'obiettivo è validare la corretta risposta del filtro Laplaciano e dell'albero di somma (Adder Tree) in presenza di variazioni di luminosità note.

#### 5.1.1 Generazione degli Stimoli

L'immagine di test è costruita alitmicamente all'interno del processo di generazione del Testbench. Essa rappresenta un quadrato chiaro su uno sfondo scuro, con i seguenti livelli di intensità (valori a 8 bit):

- **Sfondo** ( $I_{bg}$ ): Valore costante pari a **10** ( $0A_{hex}$ ).
- **Oggetto** ( $I_{fg}$ ): Un quadrato centrale (righe 10-22, colonne 10-22) con valore costante pari a **100** ( $64_{hex}$ ).

Questa configurazione permette di testare la risposta del filtro nelle regioni uniformi, lungo i bordi e in corrispondenza degli angoli.

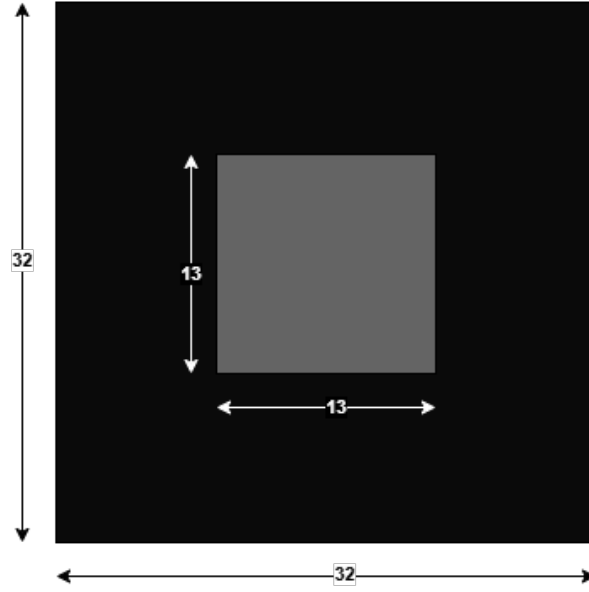


Figura 5.1: Immagine generata tramite il testbench per testare il progetto

### 5.1.2 Analisi Teorica dei Risultati Attesi

Il filtro implementato è un Laplaciano a 4 vicini, descritto dalla maschera di convoluzione  $K$ :

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (5.1)$$

L'operazione matematica eseguita per ogni pixel  $(x, y)$  è:

$$OUT(x, y) = 4 \cdot P(x, y) - (P_{su} + P_{giu} + P_{sx} + P_{dx}) \quad (5.2)$$

Di seguito si riporta il calcolo analitico dei valori attesi nelle diverse regioni dell'immagine generata.

#### 1. Regioni Uniformi (Zone Piatte)

Nelle aree in cui il pixel centrale e i suoi 4 vicini hanno lo stesso valore (tutti 10 o tutti 100), la derivata spaziale è nulla.

- **Sfondo:**  $4(10) - (10 + 10 + 10 + 10) = 40 - 40 = 0$
- **Interno Oggetto:**  $4(100) - (100 + 100 + 100 + 100) = 400 - 400 = 0$

## 2. Bordo dell'Oggetto (Salto Positivo)

Considerando un pixel sul perimetro del quadrato (es. lato superiore, lontano dagli angoli), il pixel centrale appartiene all'oggetto (100), mentre uno dei vicini appartiene allo sfondo (10).

$$OUT = 4(100) - (10_{bg} + 100 + 100 + 100) = 400 - 310 = +\mathbf{90} \quad (5.3)$$

## 3. Bordo Esterno (Effetto Alone/Salto Negativo)

Considerando un pixel dello sfondo adiacente al quadrato, il centro vale 10, ma uno dei vicini vale 100. Questo genera un picco negativo.

$$OUT = 4(10) - (100_{fg} + 10 + 10 + 10) = 40 - 130 = -\mathbf{90} \quad (5.4)$$

## 4. Angolo dell'Oggetto (Massima Variazione)

In corrispondenza dei 4 vertici del quadrato (es. angolo in alto a sinistra), il pixel centrale (100) confina con lo sfondo su due lati (sopra e sinistra).

$$OUT = 4(100) - (10_{bg} + 100 + 10_{bg} + 100) = 400 - 220 = +\mathbf{180} \quad (5.5)$$

Questo rappresenta il valore massimo positivo generabile con l'immagine di test corrente.

### 5.1.3 Tabella Riepilogativa dei Casi di Test

La seguente tabella riassume i valori attesi che devono essere riscontrati nella waveform di simulazione.

| Zona Immagine   | Centro ( $P_{xy}$ ) | Vicini ( $P_{neigh}$ ) | Formula     | Output Atteso |
|-----------------|---------------------|------------------------|-------------|---------------|
| Sfondo Uniforme | 10                  | 10, 10, 10, 10         | $40 - 40$   | <b>0</b>      |
| Interno Oggetto | 100                 | 100, 100, 100, 100     | $400 - 400$ | <b>0</b>      |
| Bordo Oggetto   | 100                 | 10, 100, 100, 100      | $400 - 310$ | <b>+90</b>    |
| Bordo Esterno   | 10                  | 100, 10, 10, 10        | $40 - 130$  | <b>-90</b>    |
| Angolo Oggetto  | 100                 | 10, 100, 10, 100       | $400 - 220$ | <b>+180</b>   |

Tabella 5.1: Valori attesi in uscita dal filtro per l'immagine di test generata (Sfondo=10, Oggetto=100).

## 5.2 Risultati della Simulazione

### 5.2.1 Fase di Reset e Inizializzazione

Come mostrato in Figura 5.2, dopo il reset, il segnale `s_axis_tvalid` diventa alto. Tuttavia, l'uscita `m_axis_tvalid` rimane bassa per un periodo corrispondente alla latenza dei buffer di linea più la latenza della pipeline aritmetica.

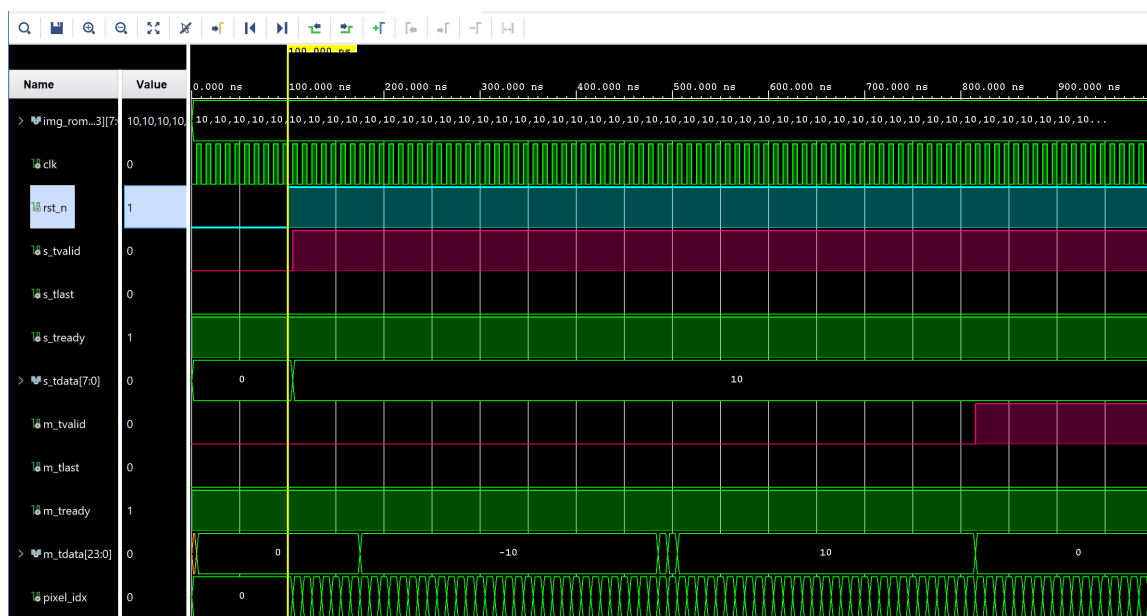


Figura 5.2: Simulazione temporale: Fase di riempimento della pipeline.

### 5.2.2 Elaborazione delle Aree Uniformi

Quando la finestra scorre sull'area di sfondo (valore 10), i 9 ingressi sono identici. Calcolo teorico:  $10 \times (4 - 1 - 1 - 1 - 1) = 0$ . La simulazione (Fig. 5.3) conferma che l'uscita dati è stabile a zero.

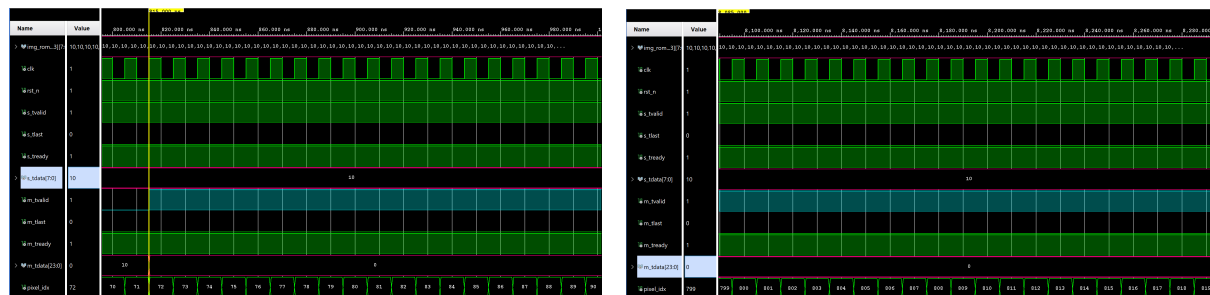
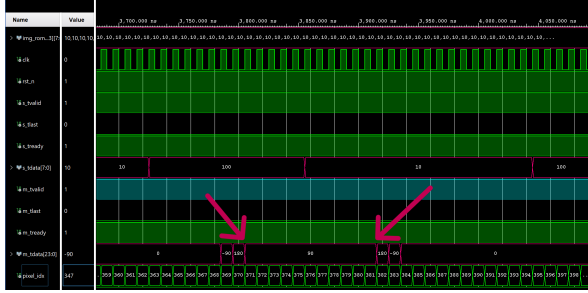


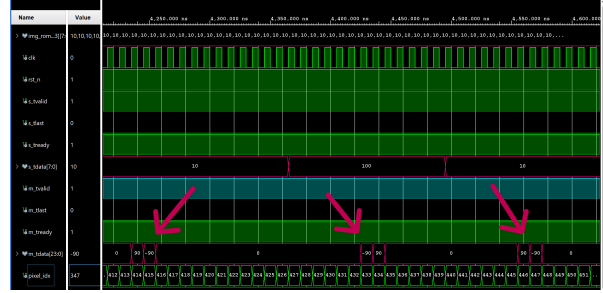
Figura 5.3: Output del filtro in aree a intensità costante.

## 5.2.3 Rilevamento dei Bordi

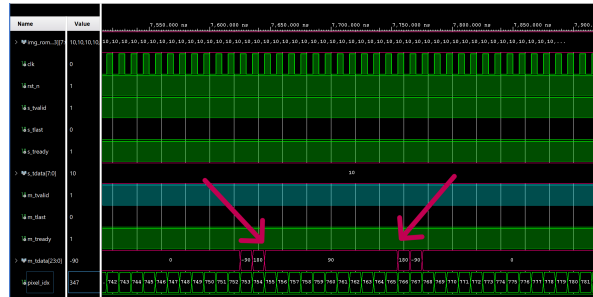
In Figura 5.4 è visibile l'istante in cui la finestra attraversa il bordo del quadrato (passaggio da 10 a 100). Si osservano picchi nel segnale `m_axis_tdata`, che confermano il corretto funzionamento del filtro come Edge Detector.



(a) Bordo superiore dell'immagine interna



(b) Bordi laterali dell'immagine interna



(c) Bordo inferiore dell'immagine interna

Figura 5.4: Rilevamento del gradino di intensità: risposta impulsiva del filtro.



# Capitolo 6

## Implementazione VHDL - Versione 2

In questa sezione, viene presentata una seconda versione dell'implementazione VHDL del filtro Laplaciano e dell'Adder Tree, con alcune ottimizzazioni rispetto alla prima versione. L'obiettivo è migliorare l'efficienza del design, mantenendo la stessa funzionalità.

### 6.1 Motivazione e Strategia di Ottimizzazione

Il kernel Laplaciano presenta una caratteristica fondamentale che non era stata sfruttata nella v1: soltanto tre valori distinti di coefficiente, ossia 0,  $-1$  e  $+4$ . Moltiplicare per 0 è un *no-op*, moltiplicare per  $+4$  equivale a uno shift sinistro di 2 bit (operazione puramente strutturale, senza logica), e moltiplicare per  $-1$  corrisponde alla negazione in complemento a 2. In questa versione si eliminano pertanto tutti e 9 i `BoothMultiplier`, sostituendoli con logica hardcoded combinatoria.

### 6.2 LaplacianFilter\_v2: Logica Hardcoded e CLA

Il blocco `LaplacianFilter_v2` riceve i 9 pixel della finestra e produce i 9 prodotti applicando direttamente le regole del kernel.

- **Coefficienti agli angoli ( $K = 0$ ):** I quattro pixel agli angoli della finestra ( $p00$ ,  $p02$ ,  $p20$ ,  $p22$ ) hanno coefficiente nullo. Il loro prodotto è pari a zero in modo immediato; di conseguenza, non è necessaria alcuna logica hardware per la loro gestione.
- **Coefficienti sui lati ( $K = -1$ ):** Per i quattro pixel sui lati ( $p01$ ,  $p10$ ,  $p12$ ,  $p21$ ) il prodotto vale  $-1 \cdot p$ . In aritmetica a complemento a 2 la negazione di un numero a  $N$  bit si ottiene invertendo bit per bit e sommando 1:

$$-p = \text{not}(p) + 1$$

Nell'implementazione VHDL il pixel a 8 bit viene esteso con zeri a 10 bit, invertito logicamente (**not**), e infine sommato a 1 tramite un **CLA\_Adder** generico con  $C_{in} = '1'$ , ottenendo un risultato a 11 bit in complemento a 2.

- **Coefficiente centrale ( $K = +4$ ):** Il pixel centrale  $p_{11}$  viene moltiplicato per +4 mediante una semplice concatenazione strutturale:

$$4 \cdot p_{11} = '0' \& p_{11} \& "00"$$

Questa operazione non richiede alcun gate logico: il valore a 11 bit risultante è ottenuto tramite cablaggio, con ritardo nullo per il percorso combinatorio.

## Carry Look-ahead Adder (CLA)

La negazione dei 4 pixel di bordo utilizza un componente **CLA\_Adder** generico di larghezza  $N$ . A differenza dell'RCA, il CLA calcola i segnali *generate* ( $G_i = A_i \cdot B_i$ ) e *propagate* ( $P_i = A_i \oplus B_i$ ) in modo parallelo, determinando il carry di ogni stadio senza attendere la propagazione in catena:

$$C_{i+1} = G_i + P_i \cdot C_i$$

La profondità logica del CLA è  $O(\log N)$  contro  $O(N)$  dell'RCA, il che in teoria ne riduce il ritardo per operandi larghi. Tuttavia, in fase di test ci si è accorti che il vantaggio teorico non si traduce necessariamente in una riduzione del ritardo fisico su FPGA.

## 6.3 AdderTree\_v2: Albero CLA

L'AdderTree\_v2 sostituisce tutti gli RCA interni con istanze **CLA\_Adder** generiche. La struttura a 4 stadi rimane invariata rispetto alla v1, con crescita dei bit da 11 a 15 bit attraverso i livelli:

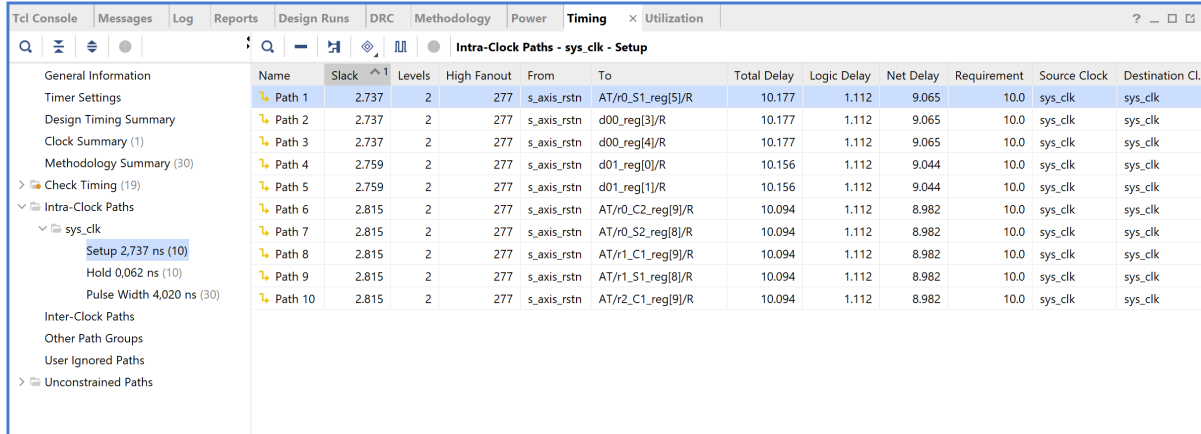
| Livello | Operazione                                                     | Larghezza uscita |
|---------|----------------------------------------------------------------|------------------|
| 1       | $p_0 + p_1, \quad p_2 + p_3, \quad p_4 + p_5, \quad p_6 + p_7$ | 12 bit           |
| 2       | somma coppie di livello 1                                      | 13 bit           |
| 3       | somma coppie di livello 2                                      | 14 bit           |
| 4       | somma finale + allineamento $p_8$                              | 15 bit           |

L'uscita dell'AdderTree\_v2 è ridotta a 15 bit (da 20 della v1), poiché il range dinamico del risultato con coefficienti  $\{0, -1, +4\}$  e pixel a 8 bit non supera 11 bit con segno, rendendo superflua la larghezza di bus originale.

## 6.4 Vincoli XDC e Pblock

Durante l'implementazione della v2, Vivado distribuiva liberamente le celle del design su lati opposti del dispositivo (Fig. 6.1b), generando percorsi di routing eccessivamen-

te lunghi (Fig. 6.1a). Per vincolare il posizionamento è stato necessario introdurre un *pblock* nel file *vincoli.xdc*, il quale forza tutte le celle del design nella regione SLICE\_X100Y45:SLICE\_X120Y70 del dispositivo; Questa scelta è stata fatta in maniera tale da posizionare il design nella stessa regione in cui Vivado ha implementato la prima versione.



| Name    | Slack | Levels | High Fanout | From        | To                | Total Delay | Logic Delay | Net Delay | Requirement | Source Clock | Destination Cl. |
|---------|-------|--------|-------------|-------------|-------------------|-------------|-------------|-----------|-------------|--------------|-----------------|
| Path 1  | 2.737 | 2      | 277         | s_axis_rstn | AT/r0_S1_reg[5]/R | 10.177      | 1.112       | 9.065     | 10.0        | sys_clk      | sys_clk         |
| Path 2  | 2.737 | 2      | 277         | s_axis_rstn | d00_reg[3]/R      | 10.177      | 1.112       | 9.065     | 10.0        | sys_clk      | sys_clk         |
| Path 3  | 2.737 | 2      | 277         | s_axis_rstn | d00_reg[4]/R      | 10.177      | 1.112       | 9.065     | 10.0        | sys_clk      | sys_clk         |
| Path 4  | 2.759 | 2      | 277         | s_axis_rstn | d01_reg[0]/R      | 10.156      | 1.112       | 9.044     | 10.0        | sys_clk      | sys_clk         |
| Path 5  | 2.759 | 2      | 277         | s_axis_rstn | d01_reg[1]/R      | 10.156      | 1.112       | 9.044     | 10.0        | sys_clk      | sys_clk         |
| Path 6  | 2.815 | 2      | 277         | s_axis_rstn | AT/r0_C2_reg[9]/R | 10.094      | 1.112       | 8.982     | 10.0        | sys_clk      | sys_clk         |
| Path 7  | 2.815 | 2      | 277         | s_axis_rstn | AT/r0_S2_reg[8]/R | 10.094      | 1.112       | 8.982     | 10.0        | sys_clk      | sys_clk         |
| Path 8  | 2.815 | 2      | 277         | s_axis_rstn | AT/r1_C1_reg[9]/R | 10.094      | 1.112       | 8.982     | 10.0        | sys_clk      | sys_clk         |
| Path 9  | 2.815 | 2      | 277         | s_axis_rstn | AT/r1_S1_reg[8]/R | 10.094      | 1.112       | 8.982     | 10.0        | sys_clk      | sys_clk         |
| Path 10 | 2.815 | 2      | 277         | s_axis_rstn | AT/r2_C1_reg[9]/R | 10.094      | 1.112       | 8.982     | 10.0        | sys_clk      | sys_clk         |

(a)



(b)

Figura 6.1

Tramite il *pblock* abbiamo risolto il problema del placement, ma abbiamo ottenuto anche un'effetto collaterale positivo, ovvero, la prossimità fisica ha ridotto anche il *net delay* sui percorsi critici; In particolare sulla rete di distribuzione del reset (*s\_axis\_rstn*), che nella v1 mostrava fanout elevato con ritardi di routing dominanti.



# Capitolo 7

## Implementazione VHDL - Versione 3

Ottenuti i risultati della v2, si è tentato di migliorare le prestazioni di timing intervenendo sull'albero di somma. L'idea è stata quella di sostituire i **CLA\_Adder** con una struttura *Carry Save Adder* (CSA), mantenendo il *pblock* introdotto nella v2, e di rivedere la logica di negazione nel **LaplacianFilter\_v3** passando da un **CLA\_Adder** a un **RCA** già implementato nella v1.

### 7.1 Motivazione: Il Carry Save Adder per Somme Multi-operando

Il problema fondamentale dell'albero di somma è la necessità di combinare 9 operandi. Negli approcci precedenti (v1 con **RCA**, v2 con **CLA**) ogni stadio produceva un risultato “risolto” (carry già propagato), il che richiedeva  $\lceil \log_2 9 \rceil = 4$  stadi di sommatore completi, ciascuno con ritardo proporzionale alla larghezza dei bit.

Il **Carry Save Adder** rompe questo schema: ricevuti tre operandi  $X$ ,  $Y$ ,  $Z$ , il CSA produce due uscite (somma parziale  $S$  e carry parziale  $C$ ) *senza propagare il carry*:

$$S_i = X_i \oplus Y_i \oplus Z_i \quad (7.1)$$

$$C_i = (X_i \cdot Y_i) + (Y_i \cdot Z_i) + (X_i \cdot Z_i) \quad (7.2)$$

### 7.2 LaplacianFilter\_v3: RCA per la Negazione

Nella v3, la logica di negazione dei pixel di bordo viene realizzata tramite un **RCA**, invece del **CLA\_Adder** della v2. La scelta è motivata da considerazioni pratiche: per larghezze ridotte, Vivado ottimizza le strutture **RCA** utilizzando le *carry chain* dedicate degli slice Xilinx (**CARRY4**), ottenendo un ritardo fisico di circa 1-2 ns indipendente da  $N$ .

## 7.3 AdderTree\_v3: Albero CSA

Il cuore della v3 è l'AdderTree\_v3, che implementa un albero di somma a 4 livelli basato su componenti CSA, seguito da un sommatore RCA finale. Tutti i 9 ingressi vengono preliminarmente estesi a 15 bit con segno, adottando una larghezza uniforme che semplifica la gestione del *bit growth*.

Il tree riduce i 9 operandi a 2 in quattro livelli, ciascuno registrato da flip-flop di pipeline. Infine il sommatore RCA finale converte i vettori della somma e dei carry in un risultato binario.

# Capitolo 8

## Risultati post-implementazione e Confronto tra le Versioni

In questo capitolo vengono presentati e discussi i risultati ottenuti post-implementazione delle tre versioni del design.

### 8.1 Timing

#### 8.1.1 Sintesi dei Risultati Globali

La Tabella 8.1 raccoglie le metriche principali di timing per le tre versioni.

Tabella 8.1: Confronto timing — v1, v2, v3

| Metrica                    | v1    | v2    | v3           |
|----------------------------|-------|-------|--------------|
| WNS (ns)                   | 4,449 | 4,515 | <b>4,718</b> |
| TNS (ns)                   | 0,000 | 0,000 | 0,000        |
| Failing endpoints          | 0     | 0     | 0            |
| Total endpoints            | 794   | 529   | 793          |
| Critical path — total (ns) | 5,478 | 5,286 | <b>4,989</b> |
| Logic delay CP (ns)        | 1,557 | 1,733 | <b>1,338</b> |
| Net delay CP (ns)          | 3,921 | 3,553 | 3,651        |
| Livelli logici CP          | 7     | 6     | <b>4</b>     |
| WHS (ns)                   | 0,062 | 0,054 | 0,026        |

Il WNS cresce in maniera monotona da 4,449 ns (v1) a 4,718 ns (v3), confermando un miglioramento progressivo del margine di setup. In tutte le versioni il vincolo di 10 ns è rispettato con ampio margine, e la frequenza massima stimata passa da circa 180 MHz (v1) a circa 190 MHz (v3), calcolata mediante la formula:  $f_{max} = 1/(T_{CLK} - WNS)$ .

### 8.1.2 Analisi del Critical Path

Il percorso critico subisce un cambiamento strutturale significativo nel corso delle tre versioni (Tabella 8.2).

Tabella 8.2: Dettaglio del critical path per versione

| Versione | Sorgente     | Destinazione  | Logic delay |
|----------|--------------|---------------|-------------|
| v1       | AT/r2_g[2]/C | AT/r3_a[17]/D | 1,557 ns    |
| v2       | AT/r2_g[1]/C | AT/r3_a[11]/D | 1,733 ns    |
| v3       | AT/r3_g[3]/C | AT/resu[10]/D | 1,338 ns    |

Nelle versioni v1 e v2 il percorso critico si colloca all'interno dell'albero di somma (stadio  $r2 \rightarrow r3$ ). Nella v3 il path si sposta all'interfaccia tra l'ultimo registro CSA (**r3**) e il registro di uscita (**result**), attraverso l'RCA finale. Questo spostamento indica che il CSA ha eliminato con successo la propagazione del carry negli stadi intermedi: il collo di bottiglia è ora concentrato nel singolo sommatore finale.

Il logic delay scende a 1,338 ns in v3 (rispetto a 1,733 ns della v2) grazie alla riduzione dei livelli logici da 6 a 4. Il net delay risale leggermente (+0,098 ns rispetto alla v2); il bilancio netto rimane comunque a favore della v3 con un guadagno di -0,297 ns sul total delay.

### 8.1.3 Reset e Distribuzione del Clock

Un fenomeno rilevante osservabile nell'analisi dei top-10 path è l'evoluzione del percorso di reset (**s\_axis\_rstn**). Nella v1 tale percorso occupa 8 delle 10 posizioni peggiori, con fanout 240 e net delay di 7,300 ns; nella v2, grazie al *pblock*, scende all'ultimo posto con net delay di 6,594 ns; nella v3 mantiene questa posizione con net delay di 6,168 ns, nonostante il fanout aumenti a 277 per effetto del maggiore numero di flip-flop (313 contro 256).

### 8.1.4 Nota sul Worst Hold Slack

Il WHS diminuisce in maniera monotona nelle tre versioni: 0,062 ns  $\rightarrow$  0,054 ns  $\rightarrow$  0,026 ns. Il margine rimane positivo in tutte le versioni (nessun endpoint failing), ma la tendenza indica che ulteriori ottimizzazioni del setup potrebbero portare a violazioni di hold.



## 8.2 Utilizzo delle Risorse

Tabella 8.3: Confronto utilizzo risorse — v1, v2, v3 (risorse totali dispositivo in parentesi)

| Risorsa              | v1  | v2  | v3  | Disponibili |
|----------------------|-----|-----|-----|-------------|
| Slice LUT totali     | 135 | 117 | 144 | 53200       |
| LUT as Logic         | 119 | 101 | 128 | 53200       |
| LUT as Memory        | 16  | 16  | 16  | 17400       |
| Slice Registers (FF) | 276 | 256 | 313 | 106400      |
| Slice fisici         | 73  | 63  | 81  | 13300       |
| Bonded IOB           | 40  | 32  | 32  | 125         |
| BUFGCTRL             | 1   | 1   | 1   | 32          |

### 8.2.1 Riduzione delle risorse dalla v1 alla v2

Il passaggio dalla v1 alla v2 produce una riduzione concreta delle risorse:  $-18$  LUT e  $-20$  FF. Il dato significativo è che il `LaplacianFilter` presenta lo stesso numero di LUT in entrambe le versioni, il che indica che Vivado aveva già ottimizzato i `BoothMultiplier` durante la sintesi della v1, convergendo alla stessa soluzione logica indipendentemente dalla descrizione RTL. La riduzione delle risorse avviene solo all'interno del `AdderTree`, dunque la causa è quasi sicuramente riconducibile alla riduzione della larghezza degli operandi in ingresso all'albero, i quali sono passati da 16 a 11 bit grazie alla semplificazione delle moltiplicazioni e di conseguenza ogni stadio di CLA richiede meno LUT.

### 8.2.2 Variazioni nella v3 rispetto alla v2

L'`AdderTree_3` aumenta significativamente:  $+37$  LUT e  $+57$  FF. L'aumento delle LUT è dovuto alla logica combinatoria CSA che opera su operandi a 15 bit (dovuto al fatto che abbiamo esteso gli operandi prima delle albero solo per la comodità di gestione del bit growth) e di conseguenza si ha anche un aumento dei FF dovuto al fatto che i segnali da registrare ad ogni stadio sono due uscite da 15 bit (Sum e Carry).

Il `LaplacianFilter_v3` invece, ha un miglioramento  $-9$  LUT, grazie alla sostituzione del `CLA_Adder` con un RCA.

Tabella 8.4: Utilizzo per sotto-blocco nelle tre versioni

| Blocco                    | v1  |     | v2  |     | v3        |            |
|---------------------------|-----|-----|-----|-----|-----------|------------|
|                           | LUT | FF  | LUT | FF  | LUT       | FF         |
| LaplacianFilter           | 35  | 44  | 33  | 44  | <b>24</b> | 44         |
| AdderTree                 | 51  | 113 | 35  | 93  | <b>72</b> | <b>150</b> |
| BufferLine (FSM + buffer) | 49  | 119 | 49  | 119 | 48        | 119        |

## 8.3 Potenza

Tabella 8.5: Confronto potenza — v1, v2, v3

| Componente                 | v1          | v2           | v3           |
|----------------------------|-------------|--------------|--------------|
| Totale On-Chip (W)         | 0,109       | 0,107        | 0,107        |
| Device Static (W)          | 0,104       | 0,104        | 0,104        |
| Dinamico totale (W)        | 0,004       | 0,003        | 0,003        |
| Clock (W)                  | 0,002 (40%) | 0,002 (78%)  | 0,002 (82%)  |
| Signals (W)                | <0,001      | <0,001       | <0,001       |
| Logic (W)                  | <0,001      | <0,001       | <0,001       |
| I/O (W)                    | 0,002 (51%) | <0,001 (12%) | <0,001 (12%) |
| Temperatura giunzione (°C) | 26,3        | 26,2         | 26,2         |
| Confidence level           | Medium      | Medium       | Medium       |

La potenza totale rimane sostanzialmente invariata nelle tre versioni, con la potenza statica del dispositivo (0,104 W) che domina per il 96-97%.

La riduzione della potenza I/O da 0,002 W (v1, 51%) a meno di 0,001 W (v2-v3, 12%) è conseguenza diretta della riduzione del bus di uscita da 24 a 16 bit. L'aumento progressivo della quota clock dal 40% (v1) all'82% (v3) riflette il maggiore numero di flip-flop introdotto dal CSA tree nella v3.

## 8.4 Discussione Critica dei Risultati

### 8.4.1 Trade-off Timing vs. Area nella v3

L'architettura v3 consegue il miglior timing ( $WNS = 4,718$  ns,  $f_{max} \approx 190$  MHz) al costo di un aumento delle risorse logiche (+27 LUT, +57 FF). Questo trade-off è accettabile nel contesto del progetto; Inoltre ci aspettiamo che gestendo il *bit growth* ad ogni stadio come nelle precedenti versioni la differenza di risorse utilizzate sia minima.

### 8.4.2 Convergenza dell'Ottimizzazione in v1 e v2

Nonostante la v1 utilizzi `BoothMultiplier` e la v2 logica hardcoded, il `LaplacianFilter` passa da 35 a 33 LUT (una riduzione minima). Questo suggerisce che Vivado aveva già ottimizzato gran parte della logica dei moltiplicatori durante la sintesi della v1, convergendo a una soluzione simile indipendentemente dalla descrizione RTL. La riduzione reale di risorse tra v1 e v2 è concentrata nell' *AdderTree*, dove la larghezza ridotta degli operandi (da 16 bit a 11 bit) produce un risparmio concreto di 16 LUT e 20 FF.

### 8.4.3 Ruolo del Pblock

Il *pblock* introdotto nella v2 per risolvere il problema di placement ha prodotto come effetto collaterale una riduzione degli slice fisici da 73 a 63, grazie alla maggiore compattezza delle celle nella regione vincolata.

Sul timing, il net delay del percorso di reset scende da 7,300 ns (v1) a 6,594 ns (v2) e 6,168 ns (v3), confermando che la prossimità fisica delle celle riduce i ritardi di routing indipendentemente dall'architettura logica.

# Capitolo 9

## Conclusioni

Questo progetto ha affrontato la progettazione e l'ottimizzazione iterativa di un filtro Laplaciano  $3 \times 3$  in hardware su FPGA, partendo da un'implementazione di riferimento basata su moltiplicatori di Booth e pervenendo a un'architettura ottimizzata con logica hardcoded e CSA tree.

Il percorso evolutivo ha prodotto risultati misurabili:

- La **Versione 1** ha stabilito la linea di base funzionalmente corretta, validata dalla simulazione e dai risultati di sintesi ( $WNS = 4,449\text{ ns}$ ,  $f_{max} \approx 180\text{ MHz}$ ).
- La **Versione 2** ha introdotto la semplificazione della logica di moltiplicazione (coefficienti hardcoded) e, per necessità di placement, il vincolo di posizionamento fisico (*pblock*). La riduzione degli operandi da 16 a 11 bit ha prodotto un risparmio concreto nell'AdderTree. Come effetto collaterale del *pblock*, il timing migliora leggermente e la distribuzione del reset, precedentemente dominante nel profilo di timing, viene significativamente migliorata.
- La **Versione 3** ha tentato di migliorare ulteriormente il timing sostituendo i CLA\_Adder con un albero CSA. Il risultato è positivo lato timing, ma comporta un aumento delle risorse a causa della larghezza uniforme a 15 bit e del doppio segnale da registrare per stadio.

Il design finale soddisfa tutti i requisiti di progetto: elaborazione in streaming pixel-per-pixel con interfaccia AXI-Stream, frequenza di clock di 100 MHz rispettata con ampio margine in tutte e tre le versioni, utilizzo delle risorse inferiore all'1% del dispositivo target e consumo di potenza di 0,107 W, dominato dalla componente statica del dispositivo.